

Forced Complementarity: A Personal Account of the Sum-of-Logits Objective

Notes from my joint-ensemble experiments — August 2026

1. The Problem I Started With

Suppose we are working in a distributed learning setting. Each participant can host only a small model, while the collective performance we want is beyond what any one participant's model can achieve on its own. The most obvious solution is for everyone to train independently and then average their outputs—in other words, bagging.

The alternative I am exploring can be defined in one line:

$$\mathcal{L} = \text{CE} \left(\sum_{i=1}^k f_i(x), y \right)$$

There are k independently parameterized models, and I apply a **single** cross-entropy loss to the **sum** of their logits. There is no per-member CE, no KL or distillation term between members. At inference time, the deployed object is the same sum of logits.

It is important to be clear about what the main comparison is. **The primary competitor is not a dense model, but bagging.** If a single machine can host the large model in the first place, then the original problem setting disappears. I use the dense model instead as (a) a reference point for the upper bound on representational power, and (b) a probe for separating what we are actually measuring—capacity or optimization. As I found in Section 6, that distinction turned out to be decisive.

2. What the Objective Function Implies

The gradient of the sum-of-logits objective is surprisingly simple. For member i 's logits z_i ,

$$\frac{\partial \mathcal{L}}{\partial z_i} = \text{softmax} \left(\sum_j z_j \right) - \text{onehot}(y)$$

Every member receives exactly the same signal. And that signal is not an evaluation of its own output; it is the residual of the ensemble as a whole.

That single fact explains almost everything I observe later. The members are never told, “get the answer right.” They are told only, “make up whatever the current sum is missing.” If one member is already pushing sufficiently in the right direction, the remaining gradient does not necessarily point in that same direction for the other members. Individual accuracy is not part of the objective, so optimization has no reason to preserve it.

To place this relative to neighboring methods:

Method	Signal received by each member	Communication between members	Training order
Bagging	CE against the target	None	Simultaneous / independent
Codistillation	Target CE + mutual KL	Shared output distributions	Simultaneous
Boosting	Residual from the previous stage	Sequential (frozen previous stages)	Sequential
Sum-of-logits	Ensemble residual (identical)	Only through the summed logits	Simultaneous

Sum-of-logits is conceptually similar to boosting in that it learns residuals, but it is not sequential. It is simultaneous like codistillation, but there is no explicit communication term. The members interact only through the loss function.

There is earlier work that looks at this spectrum from another angle. Negative Correlation Learning (NCL, Liu & Yao, 1999) parameterizes roughly the middle of this spectrum by mixing individual CE with an ensemble-diversity term using a coefficient λ . With $\lambda = 0$, you get bagging; with $\lambda = 1$, you get something close to the sum-of-logits setup in this report. In that sense, these experiments revisit one extreme of the NCL spectrum ($\lambda = 1$, although not exactly identical because this uses CE-on-sum rather than regression) at CIFAR/MNIST scale. NCL itself was primarily validated on regression problems, and systematic reproduction at classification/deep-learning scale appears relatively uncommon, which gives these results a complementary place in that literature.

3. It Works: A Consistent Advantage over Bagging

Across three very different kinds of experiments, I got results pointing in the same direction.

CIFAR-100 / ResNet-20 × 3 / 10 epochs

Condition	Test accuracy	NLL	ECE
Single model	49.11%	1.862	1.36%
Independent ensemble (average)	53.00%	1.691	2.59%
Jointly trained sum-of-logits	55.48%	1.577	2.40%

That is a +2.48 percentage-point gain over bagging. There is an immediate caveat, though. Two runs with the same seed, in [results/](#) and [diagnostics/](#), produced 54.93% and 55.48%, respectively. Because of nondeterministic kernels and [torch.compile](#), fixing the seed does not guarantee perfect reproducibility. The run-to-run variation is around 0.5 percentage points, while the gap over bagging was more than 2.1 points in both runs. The gap is larger than the noise, but the noise is not zero.

(Looking at NLL and ECE side by side follows the same general evaluation practice used in the deep-ensemble uncertainty literature—e.g. Lakshminarayanan, Pritzel & Blundell, 2017. The difference is that their work focuses on calibration gains from independently trained, bagging-style ensembles, whereas here the interesting trade-off is that joint training improves accuracy while changing calibration behavior—in this particular result, ECE actually improves slightly, from 2.59% to 2.40%.)

MNIST / width-2 MLP (784→2→10) / Scaling the Number of Members

I pushed the bottleneck to an extreme: each member had only two hidden units, and then I increased K .

K	Independent (bagging)	Sum-of-logits
1	68.90%	69.92%
2	72.39%	78.30%
3	80.57%	86.97%
5	80.14%	91.89%
8	80.94%	94.10%
16	80.52%	96.11%

This is the strongest picture in the whole set of experiments. **Bagging essentially saturates at $K = 3$.** It stays around 80–81% no matter how many more members I add. The independently trained bottleneck models make similar mistakes, so averaging more of them

does not change much. Sum-of-logits, on the other hand, keeps improving monotonically with K , reaching 96%.

Random Vector Memorization (AdamW)

I used 64-dimensional Gaussian vectors with random labels. I compared an 8-member ensemble with 11,856 parameters against a parameter-matched dense model with 11,791 parameters. The following are the average training accuracies over three runs:

Number of vectors	Single	Bagging	Sum-of-logits	Dense
2,048	51.3%	94.1%	100.0%	100.0%
8,192	23.3%	60.1%	83.8%	79.0%

The three experiments differ in data, architecture, and even the nature of the task—real image classification, an extreme bottleneck, and pure memorization—but they all point in the same direction when it comes to the advantage over bagging.

4. Why Does It Work? The Members Give Up on Being Classifiers

This is where the gradient observation from Section 2 shows up in the measurements.

Looking at the member-level diagnostics from the CIFAR-100 run:

Diagnostic	Independent ensemble	Sum-of-logits
Mean individual member accuracy	49.44%	15.58%
Individual member accuracies	49.13 / 49.24 / 49.94	7.64 / 11.31 / 27.78
Correct-vs-best-wrong logit margin	-0.06 / -0.03 / +0.00	-2.00 / -1.93 / -1.35
Logit L2 norm (mean)	34.3 / 33.2 / 34.0	14.9 / 16.1 / 26.5
Pairwise logit correlation (overall)	0.91	0.06

Pairwise logit correlation (correct class)	0.91	-0.19
Ensemble accuracy	53.00%	55.48%

The sum-of-logits members are **almost useless as standalone classifiers**. On a 100-class problem, 7.64% is better than random chance (1%), but it is hard to call that a classifier. The fact that all the margins are negative means that, on average, an individual member does not even give the highest logit to the correct class.

Two things stand out in particular.

First, **the correlation of the correct-class logits is negative (-0.19)**. The overall logit correlation of 0.06 might be read simply as “the members became uncorrelated,” but when I look only at the correct class, the members are actually moving in **opposite directions**. When one member pushes the correct class for a particular sample, another member pulls it back. This looks less like accidental decorrelation and more like division of labor.

Second, **the logit norms are less than half as large** (34 versus 15–26). Each member produces a relatively small vector, and only their sum reaches the scale of a conventional model. A member is not trying to make a complete prediction; it is contributing a piece.

I would call this **forced complementarity**.

In bagging, diversity is largely an incidental consequence of initialization and data ordering, which is why the members remain highly correlated at 0.91. In sum-of-logits training, diversity is produced directly by the objective. Since individual accuracy is not something the objective has any reason to preserve, the members can sacrifice it and gain complementarity instead.

The basic idea of directly forcing diversity through the loss is not new. Krogh & Vedelsby (1994)'s ambiguity decomposition showed that ensemble error can be decomposed exactly into average individual error and a diversity, or ambiguity, term. Liu & Yao (1999)'s NCL explicitly penalizes this diversity-related correlation during training. Brown & Wyatt (2003) formally showed that NCL can be understood as a derivative of ambiguity decomposition. The observation here—that correct-class logit correlations become negative—is therefore consistent with the direction predicted by this line of theory. The difference is that this setup does not introduce a separate diversity weight λ ; it obtains diversity as a byproduct of applying CE directly to the summed logits. That makes the formulation simpler, but it also means there is no explicit knob for controlling how much diversity is encouraged.

How Should I Read Correlations Going to Zero?

In the width-2 MNIST experiments, the correlation of the sum-of-logits members moves from -0.32 toward 0.01 as K increases. It would be wrong to interpret this as “diversity disappears as the number of members increases.”

If K variables all have the same pairwise correlation ρ , positive semidefiniteness of the covariance matrix imposes a structural lower bound:

$$\rho \geq -\frac{1}{K-1}$$

So at $K = 16$, even perfect anti-correlation cannot go below -0.067.

Normalizing the observations relative to this lower bound gives:

K	Lower bound $-1/(K-1)$	Observed correlation	Fraction of lower bound
2	-1.000	-0.319	32%
3	-0.500	-0.244	49%
5	-0.250	-0.115	46%
8	-0.143	-0.040	28%
16	-0.067	+0.012	-18%

A significant part of the movement toward zero in absolute terms comes from the lower bound itself moving toward zero. At the same time, the fraction of that bound also declines after peaking at 49% for $K = 3$, so it would be too strong to say that the pressure toward anti-correlation remains unchanged all the way through.

The precise statement is this: **a decrease in the absolute value of correlation is not evidence that diversity is disappearing; much of it is a consequence of structural constraints.** And accuracy continues to rise throughout this range, from 86.97% to 96.11%. The performance metric helps correct how the diversity metric should be interpreted.

5. How Does Complementarity Emerge? Symmetry Is a Fixed Point, but an Unstable One

For the members to take on different roles, they have to become different in the first place. But as we saw in Section 2, every member receives **exactly the same gradient**. So what happens if the members start from exactly the same initialization? Would they remain identical forever?

Yes. And that turns out to be the only collapse mode.

I tested 2,048 random vectors with 8 members for 800 AdamW steps:

Initialization	Ensemble accuracy	Parameter cosine	Logit correlation	Prediction disagreement
Independent random	100.0%	0.009	-0.087	92.40%
Shared base + 0	41.57%	1.000	1.000	0.00%
Shared base + $1e^{-5}$	100.0%	0.360	-0.064	91.24%
Shared base + $1e^{-4}$	100.0%	0.317	-0.064	91.62%
Shared base + $1e^{-3}$	100.0%	0.285	-0.060	91.12%
Shared base + $1e^{-2}$	100.0%	0.169	-0.094	93.36%
Shared base + $1e^{-1}$	100.0%	0.045	-0.073	92.15%

Exact symmetry is an **invariant**. Identical parameters receive identical gradients and therefore remain identical. The eight members effectively become one model scaled eight times, which gives only 41.57%.

But this fixed point is **unstable**. A perturbation as small as 10^{-5} is enough to break it. Under that condition, parameter cosine similarity falls from 1.000 at step 0 to 0.688 after 100 steps and 0.360 after 800 steps. Logit correlation moves from 1.000 to 0.129 to -0.064. Tiny differences at initialization are amplified by the joint objective and spread the members into low-correlation functions.

In practical terms, I would summarize this as follows: **collapse is not a failure mode I need to worry much about**. With standard random initialization, the system automatically avoids it. The more interesting implication is the opposite: even if participants in a distributed setting start from the same pretrained checkpoint, injecting a tiny amount of noise may be enough for joint training to automatically differentiate their roles.

6. The Turning Point: What I Was Measuring Was Optimization, Not Capacity

The random-vector result in Section 3 initially looked like this: at 8,192 points, sum-of-logits reached 83.8%, beating the dense model's 79.0%. Since the parameter counts were almost identical, it was tempting to conclude that “distributed small models can represent more than a dense model with the same parameter budget.”

Then I changed the optimizer to full-batch L-BFGS with a strong-Wolfe line search, and the conclusion flipped.

Number of vectors	Single	Bagging	Sum-of-logits	Dense
2,048	52.88%	98.49%	100.00%	100.00%
8,192	24.51%	54.97%	86.99%	100.00%

The dense model memorized all 8,192 points **completely** (NLL 4.7×10^{-8}). Sum-of-logits stopped at 86.99%. The 79% I observed for the dense model with AdamW was not a capacity limit; it was simply **a point the optimizer failed to reach within that optimization budget**.

This does not invalidate the claim from Section 3. The advantage over bagging remains under L-BFGS (54.97% vs. 86.99%), and the gap is actually larger. What breaks is the **claim I should never have made in the first place**: that a distributed ensemble beats an equally sized dense model in representational capacity.

This is why I insisted in Section 1 that bagging is the real competitor.

The lesson is methodological. A capacity comparison measured with a single minibatch first-order optimizer is not really a capacity comparison. It measures the interaction between the optimizer and the architecture. Why sums of bottlenecked models might be easier to optimize with a first-order method is a separate question—an interesting one, but not one this experiment answers.

The Cost of Complementarity: What Happens When One Member Disappears?

Forced complementarity comes with a cost.

When I remove one member from the CIFAR-100 ensemble, the accuracy becomes:

Condition	Full ensemble	After removal (worst case)
Independent ensemble	53.00%	51.85% (-1.15 pp)
Sum-of-logits	55.48%	20.22% (-35.26 pp)

In bagging, every member is a complete classifier, so losing one does not prevent the others from working. In sum-of-logits, a member is only a partial contributor. Remove it, and the remaining members may be left partially cancelling one another.

In a distributed-system framing, that is a serious property: if one participant drops out, the entire system can collapse.

But this is a $K = 3$ story, and **the effect depends strongly on K** .

The width-2 MNIST leave-one-out results are:

K	Ensemble	LOO mean	Change
2	78.30%	27.56%	-50.74 pp
3	86.97%	45.50%	-41.47 pp
5	91.89%	66.81%	-25.08 pp
8	94.10%	80.59%	-13.51 pp
16	96.11%	92.76%	-3.35 pp

The vulnerability decreases monotonically with K . At $K = 16$, losing one member costs only 3.35 percentage points, and the resulting 92.76% is still far above the 80.52% achieved by bagging under the same conditions.

So the vulnerability can largely be mitigated by having enough members. The structure favors **many weak participants over a small number of strong ones**, which happens to fit the distributed scenario I had in mind from the beginning.

The imbalance is also visible in the $K = 3$ CIFAR run. Individual member accuracies are 7.64 / 11.31 / 27.78, while their leave-one-out accuracies are 42.65 / 41.56 / 20.22. The third member is much more important than the other two. The solution is complementary, but **not evenly complementary**. Whether this imbalance should be explicitly suppressed with a regularizer remains an open question.

7. Two Design Choices

Where Should the Models Be Combined: Logits or Hidden States?

CIFAR-100, ResNet-18 (width multiplier 16), four members, vectorized with grouped convolution, 20 epochs.

- **logit_sum**: each member has its own classification head, $\sum_i W_i h_i$
- **hidden_sum**: sum the hidden vectors first, then use one shared head, $W \sum_i h_i$

Condition	Parameters	Test accuracy	Raw NLL	Calibrated NLL
Logit sum	2,852,304	65.33%	1.2254	1.2095
Hidden sum	2,813,604	63.27%	1.3072	1.2924

Logit combination is ahead by 2.06 percentage points. Since the parameter counts are not exactly matched—a 1.4% difference—this result is not decisive.

One possible interpretation is that a shared head has to decode the combined representation, whereas with logit summation each member is already speaking in the class space. From a distributed-systems perspective, logit summation also requires far less communication: the number of classes rather than the hidden dimension.

Simultaneous vs. Sequential: Additive Logit Correction

What happens if I build the same sum sequentially?

At stage t ,

$$z_t(x) = \text{stopgrad}(z_{t-1}(x)) + f_t(x)$$

All previous stages are frozen, and only the new network is trained.

For random-label memorization, using a **64→16→16→10** correction network, averaged over three runs:

Number of vectors	Stage 1	Stage 2	Stage 4	Stage 8
2,048	49.90%	78.66%	99.93%	100.00%
8,192	23.01%	33.24%	51.25%	81.45%

The simultaneous joint-training result of 83.8% (AdamW, 8 members) and the 81.45% from eight sequential stages are fairly close. Given that the final object is the same sum of eight logits, that makes sense.

The sequential version has one practical advantage—it requires only one network to be trained at a time, which reduces memory requirements—but it also has a major drawback: once a previous stage makes a mistake, later stages cannot revise it.

This procedure is essentially a special case of gradient boosting (Friedman, 2001). Friedman's formulation generalizes boosting to steepest descent in function space for arbitrary differentiable losses, where each stage fits the residual gradient of the sum of all previous stages. The equation $z_t = \text{stopgrad}(z_{t-1}) + f_t$ is exactly that form. The main differences are that the weak learner here is a neural network rather than a tree, and there is no explicit shrinkage corresponding to a learning rate. The monotonic increase from Stage 1 to Stage 8 looks very familiar from the boosting literature.

I also ran an experiment with dropout $p = 0.7$ on the MNIST 2D bottleneck, pushing the effective capacity of each correction network to an extreme. With dropout disabled at inference, accuracy rose from 44.71% at Stage 1 to 92.30% at Stage 8.

Whether the frozen previous logits were computed deterministically or sampled with dropout had almost no effect on final accuracy (92.30% vs. 92.05%), although the deterministic baseline was clearly better in NLL (0.3688 vs. 0.5099).

8. Looking at It as Distributed Learning

I motivated this objective from a distributed-learning setting in Section 1, but all of the experiments so far have actually been run on a single machine.

This section tries to close that gap.

One important qualification: **the communication numbers below are analytical estimates derived from the architecture and objective, not measurements.** I have not actually run the system across multiple nodes. I summarize what should be measured at the end.

What Gets Communicated?

The amount of information exchanged between participants in the sum-of-logits objective is surprisingly small.

At every step:

1. Each member sends its logits $z_i \in \mathbb{R}^{B \times C}$ to the aggregator (uplink).
2. The aggregator computes $g = \text{softmax}(\sum_j z_j) - \text{onehot}(y)$.
3. The **same** $g \in \mathbb{R}^{B \times C}$ is broadcast to every member (downlink).
4. Each member receives g and backpropagates through its own network.

The gradient symmetry from Section 2 translates directly into a communication advantage. There is no need to calculate and transmit a different signal for every member. One tensor can simply be broadcast to everyone.

The key point is that **communication volume is completely independent of model size**. It is $O(BC)$ rather than $O(P)$.

This is worth comparing with codistillation (Anil et al., 2018, “Large Scale Distributed Neural Network Training Through Online Distillation”). Codistillation also aims to separate communication volume from parameter count by exchanging predictions—logits or probability distributions—rather than entire models. But their implementation uses stale checkpoints that are read relatively infrequently, with each worker training independently on its own subset of data and adapting to the other workers’ delayed predictions. Sum-of-logits is more tightly coupled: (a) the logits themselves enter the loss directly, and (b) complete synchronization is required at every step. If codistillation is “loose consensus,” this is closer to “jointly computing a single objective.” The synchronization cost discussed below is also exactly the problem codistillation was designed in part to avoid.

Comparing the Scale

For the CIFAR-100 setup ($B = 384$, $C = 100$, $K = 3$, fp16):

Method	Traffic per member per step	Dependence on model size
Bagging	0	—
Sum-of-logits	150 KB (75 up + 75 down)	None
Dense data parallelism	742 KB (ring all-reduce)	$O(P)$

The dense reference assumes a ResNet-20 with 278,324 parameters synchronized through ring all-reduce, using $2 \frac{K-1}{K} P$ bytes per worker. Sum-of-logits is about one-fifth as much.

But the ratio itself is not the most important point. What matters is that the two terms scale differently:

$$\frac{\text{sum-of-logits}}{\text{data parallelism}} = \frac{B \cdot C \cdot K}{(K - 1) \cdot P}$$

For ResNet-20, the ratio is 0.21. For a model with 1B parameters, it would fall to roughly 10^{-4} . On the other hand, the advantage disappears when the number of classes is extremely large—for example, when the output dimension is comparable to a vocabulary size.

The communication advantage of sum-of-logits grows as the parameter count becomes large relative to the output dimension. That is the regime most practical models tend to occupy.

Over an entire epoch, 45,000 samples / batch size 384 gives 117 steps, or 17.6 MB per member. Over 20 epochs, that is 351 MB. Even in absolute terms, that does not look particularly burdensome.

The Problem Is Latency, Not Bandwidth

The communication calculation above shows only the optimistic half of the picture.

The real constraint is **where synchronization occurs**.

In data-parallel training, gradient all-reduce can overlap with backpropagation. While earlier layers are being backpropagated, gradients for later layers can already start being transmitted. Communication can be hidden behind computation.

That does not work here.

With sum-of-logits, the forward pass has to **stop at the logits**, and backpropagation cannot begin until the round trip is complete. Communication therefore sits directly on the critical path. During that wait, every member also has to keep its activations in memory.

As a rough estimate, suppose a round trip takes 100 ms. With 20 epochs $\times$ 117 steps = 2,340 steps, that gives 234 seconds of pure waiting time. That may be tolerable for a small experiment, but it scales linearly with the number of steps and cannot simply be hidden behind computation.

The usual mitigations apply: increase the batch size to reduce the number of steps, or use microbatch pipelining so that communication for one microbatch can be hidden behind the forward pass of the next.

There is also the **straggler problem**. Since all members synchronize at every step, the step time is determined by the slowest member. As K grows, the expected maximum also grows. Bagging has a fundamental advantage here because this coupling does not exist at all.

The Sequential Variant Is Dramatically Better for Communication

In Section 7 I treated additive logit correction mainly as an alternative between simultaneous and sequential training. From a distributed perspective, however, it represents a completely different communication regime.

At stage t , the participant **does not communicate with anyone during training**. All it needs is the frozen z_{t-1} , which can be computed once over the training data and cached.

The entire training set requires only one $N \times C$ tensor per stage.

Method	Total transfer (CIFAR-100, 8 stages / 8 members)	Synchronization
Sum-of-logits (20 epochs)	351 MB per member	Every step
Additive correction	9 MB per stage, once	None (fully asynchronous)

That is a 40× difference. More importantly, there is **no round-trip latency and no straggler problem**. Participants can connect at different times, and a slow participant does not hold up a fast one.

The cost is performance. On 8,192 random vectors, simultaneous joint training reaches 83.8% (AdamW, 8 members), while eight sequential stages reach 81.45%. The sequential method also cannot undo mistakes made by earlier stages.

This is probably the most practically useful trade-off that came out of the project.

If synchronized communication is affordable, use joint training. If participants are intermittent and the network is poor, use sequential correction. There is probably a useful middle ground somewhere between the two—for example, synchronizing only once every n steps—but I have not tested it yet.

What This Framing Requires—and What It Doesn't

It does not require architectural homogeneity.

The participants only need to agree on the output dimension C and the batch construction. Their internal architectures can be completely different. A ResNet and an MLP can be mixed without changing the objective. That could be a substantial advantage when participants have heterogeneous hardware.

It does require shared data.

This is the decisive constraint.

To compute $\sum_i f_i(x)$, every member has to run the **same x in the same order** through its network. So this is **not federated learning in the usual sense**, where participants hold different local datasets. The data is shared; what is divided is model capacity.

This is still a real scenario—for example, when everyone wants to train on a public dataset but nobody can host the full model. But it does not apply to the case where data cannot be centralized for privacy reasons.

There is also a model-privacy issue. Sharing sample-level logits at every step exposes a substantial amount of information about each participant's model function. Since the data is already shared, this is not a data-privacy problem, but there is effectively no model privacy.

Dropout Robustness, Revisited

The leave-one-out vulnerability from Section 6 is exactly the participant-dropout problem in a distributed system.

In summary:

- $K = 3$: losing one member can cost as much as -35.26 percentage points. In practice, the system collapses.
- $K = 16$: losing one member costs only -3.35 points, and the post-dropout performance of 92.76% is still far above the 80.52% achieved by bagging under the same conditions.

So the system becomes more robust to dropout as K grows large enough.

The structure favors many weak participants over a small number of strong ones, which is consistent with the distributed scenario I originally had in mind.

There is one important caveat: in a real system, participants may drop out during training. All of the numbers above remove a member **after training has finished**. I have not measured what happens when a participant disappears midway through training and the remaining members have to adapt.

The structure of the objective suggests that they should adapt—the remaining members would receive new residuals corresponding to the missing contribution—but this still needs to be tested.

What I Still Need to Measure

Since this entire section is analytical, I want to leave behind a concrete validation checklist.

1. **Actual multi-node wall-clock time.** Do the latency estimates hold, and how much can pipelining hide?
 2. **Relaxed synchronization.** If synchronization happens every n steps instead of every step, and stale logits are used in between, how much accuracy is lost? If this works, the communication regime changes completely.
 3. **Participant dropout/join during training.** Not static leave-one-out, but a genuinely dynamic scenario.
 4. **Mixed architectures.** Theoretically possible, but I have not actually tested it.
 5. **Tasks with large C .** Where exactly does the communication advantage disappear?
-

9. What Remains, and What Needs Caution

There is one item that needs correction.

My initial notes presented `joint_mean`—applying CE to the mean rather than the sum—as a comparison condition and treated its ECE of 29.95% as a problem. That was a misreading.

In the implementation, the mean condition divides the logits by k only in the training loss, while evaluation uses the sum. The high ECE was therefore not a real finding; it was an **evaluation-scale mismatch artifact**.

After temperature scaling, the fitted temperature is $T = 2.91 \approx k$, and ECE drops to 1.58%, putting it in the same range as the other conditions.

The real differences are accuracy (49.94% vs. 55.48%) and calibrated NLL (1.809 vs. 1.573). The underlying reason is that dividing the logits by k also scales each member's gradient by $1/k$.

So this is not merely a question of output-scale control. It is effectively **learning-rate control**. A proper comparison would need a condition where the learning rate is scaled by k .

Remaining Limitations

- The CIFAR and MNIST results are single-seed pilots. Since I observed about 0.5 percentage points of variation across repeated runs with the same seed, making a claim about a 2-point gap requires multiple seeds.
- The L-BFGS random-vector result was run only once.
- Random-label memorization measures optimization and capacity, which is a different axis from generalization on useful data.
- The `logit_sum` vs. `hidden_sum` comparison is not parameter-matched.
- **All of the communication analysis in Section 8 is derived rather than measured.** Every experiment was run on a single machine. The bandwidth calculations are reasonably reliable, but the actual magnitude of latency, straggler effects, and pipelining benefits cannot be known without a multi-node experiment. The five items at the end of Section 8 are the next experiments I would run.

- Joint training assumes that all participants see **the same data in the same order**. This is different from conventional federated learning, and that condition should have been stated explicitly in Section 1.

In the End

The sum-of-logits objective does not ask each member to perform well individually. It shows each member only the residual of the ensemble.

As a result, a member gives up being a standalone classifier and instead becomes a complementary partial contributor.

That **forced complementarity** appears to be the reason for the consistent advantage over bagging (Sections 3 and 4). It emerges automatically once initialization symmetry is broken (Section 5), and it is also the source of the vulnerability to member dropout (Section 6).

These are three faces of the same mechanism.

From the distributed perspective, the structural advantage is that communication is $O(BC)$ and independent of model size. The structural cost is that synchronization sits directly on the critical path.

Sequential additive correction almost eliminates that synchronization cost, at the price of some performance. The most practical next step is probably to explore the space between these two extremes.

The claim that this distributed ensemble can beat a dense model with the same resource budget did not survive a change of optimizer. What remains is narrower, but also much more robust:

When participants have access to the same data but can host only small models, joint training can extract substantially more collective capacity than bagging.

References

These are not citations for direct claims made from prior literature. They are references I added after the fact to places where I found myself thinking, “I have seen something like this before.” The experiments themselves were conducted independently; these references are mainly for situating the observations rather than claiming a validated comparison with prior work.

- Liu, Y. & Yao, X. (1999). “Ensemble Learning via Negative Correlation.” *Neural Networks*, 12(10), 1399–1404. **Negative Correlation Learning (NCL)**. An early work that explicitly parameterizes something like the spectrum between bagging and sum-of-logits by mixing individual CE with a penalty that reduces correlation with the

ensemble average. The $\lambda = 1$ extreme is the closest point of comparison to this setup.

- **Krogh, A. & Vedelsby, J. (1994).** “Neural Network Ensembles, Cross Validation, and Active Learning.” *NIPS* 7, 231–238. **Ambiguity decomposition.** Ensemble error = average individual error – diversity (ambiguity). This provides the theoretical background for interpreting the negative correct-class logit correlations observed in Section 4.
- **Brown, G. & Wyatt, J. L. (2003).** “The Use of the Ambiguity Decomposition in Neural Network Ensemble Learning Methods.” *ICML*. A formal connection between NCL and ambiguity decomposition, providing a bridge for viewing NCL and this approach within the same theoretical framework.
- **Friedman, J. H. (2001).** “Greedy Function Approximation: A Gradient Boosting Machine.” *Annals of Statistics*, 29(5), 1189–1232. Generalizes stagewise additive modeling as steepest descent in function space. The additive logit correction in Section 7, $z_t = \text{stopgrad}(z_{t-1}) + f_t$, is essentially a special case of this framework, with neural networks instead of trees and no explicit shrinkage.
- **Anil, R., Pereyra, G., Passos, A., Ormandi, R., Dahl, G. E. & Hinton, G. E. (2018).** “Large Scale Distributed Neural Network Training Through Online Distillation.” *ICLR*. **Codistillation.** Shares the goal of separating communication from parameter count by exchanging predictions rather than parameters, but uses a much looser form of coupling based on stale checkpoints. This provides a useful contrast with the per-step synchronization required here.
- **Lakshminarayanan, B., Pritzel, A. & Blundell, C. (2017).** “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles.” *NeurIPS*. A standard reference for calibration and uncertainty estimation with independently trained, bagging-style ensembles. The practice of reporting NLL and ECE together in Section 3 is related to this evaluation tradition.

Intentionally Not Discussed

Federated learning (McMahan et al., 2017, FedAvg, etc.) appears in Section 8 only as a contrast.

The key distinction is that conventional federated learning distributes the **data**, whereas this setup distributes **model capacity**. Treating federated learning itself as a directly related line of work would therefore risk creating more confusion than clarity, so I have intentionally left it out of the main discussion.